

NP-Completeness

Fundamental Algorithms

Rodica Potolea, Camelia Lemnaru and Ciprian Oprea

Technical University of Cluj-Napoca
Computer Science Department

Agenda

- 1 Complexity classes
- 2 Classic NPC problems
- 3 Dealing with NPC problems in real world



Agenda

- 1 Complexity classes
- 2 Classic NPC problems
- 3 Dealing with NPC problems in real world



Polynomial problems (P)

Definition

The class **P** consists of problems that are solvable in polynomial time. The complexity is $O(n^k)$ for some constant k where n is the size of the input.



Polynomial problems (P)

Definition

The class **P** consists of problems that are solvable in polynomial time. The complexity is $O(n^k)$ for some constant k where n is the size of the input.

- most problems studied in this course are polynomial
 - sorting algorithms
 - balancing binary trees
 - minimum spanning trees
 - shortest paths in graphs
 - ...



Polynomial problems (P)

Definition

The class **P** consists of problems that are solvable in polynomial time. The complexity is $O(n^k)$ for some constant k where n is the size of the input.

- most problems studied in this course are polynomial
 - sorting algorithms
 - balancing binary trees
 - minimum spanning trees
 - shortest paths in graphs
 - ...
- problems solvable in polynomial time are also called **tractable**



Polynomial problems (P)

Definition

The class **P** consists of problems that are solvable in polynomial time. The complexity is $O(n^k)$ for some constant k where n is the size of the input.

- most problems studied in this course are polynomial
 - sorting algorithms
 - balancing binary trees
 - minimum spanning trees
 - shortest paths in graphs
 - ...
- problems solvable in polynomial time are also called **tractable**
- most practical polynomial problems have small values for k
 - we don't usually encounter $O(n^{100})$ algorithms
 - usually $k \leq 5$



Non-polynomial problems

- there are problems that are **definitely** not solvable in polynomial time
 - list all the subsets of a given set
 - list all the permutations for a given sequence



Non-polynomial problems

- there are problems that are **definitely** not solvable in polynomial time
 - list all the subsets of a given set
 - list all the permutations for a given sequence
- for the problems above, the output is exponential ($O(2^n)$) or even factorial ($O(n!)$), no polynomial algorithm could produce it



Non-polynomial problems

- there are problems that are **definitely** not solvable in polynomial time
 - list all the subsets of a given set
 - list all the permutations for a given sequence
- for the problems above, the output is exponential ($O(2^n)$) or even factorial ($O(n!)$), no polynomial algorithm could produce it
- there are also problems whose output is polynomial but are still **intractable**



Non-polynomial problems

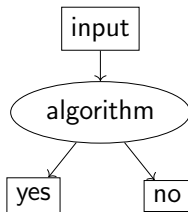
- there are problems that are **definitely** not solvable in polynomial time
 - list all the subsets of a given set
 - list all the permutations for a given sequence
- for the problems above, the output is exponential ($O(2^n)$) or even factorial ($O(n!)$), no polynomial algorithm could produce it
- there are also problems whose output is polynomial but are still **intractable**
- there are also **undecidable problems**, that cannot be solved by any algorithm
 - *Turing's halting problem*: decide if an algorithm terminates for a given input



Decision and optimization problems

Decision problems

- can be stated as a yes-no question





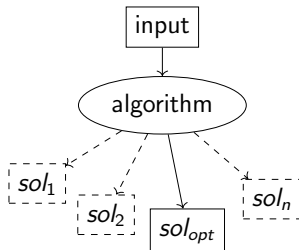
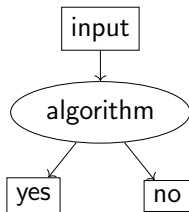
Decision and optimization problems

Decision problems

- can be stated as a yes-no question

Optimization problems

- find the best output for a given input





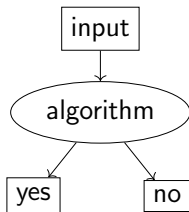
Decision and optimization problems

Decision problems

- can be stated as a yes-no question

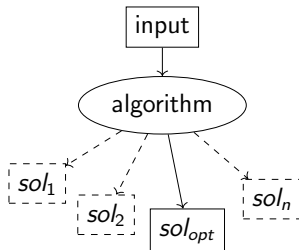
Optimization problems

- find the best output for a given input



Example:

- decision: in a given graph, is there a path shorter than k edges?
- optimization: find a shortest path in a given graph





Nondeterministic polynomial problems (NP)

Definition

The class **NP** consists of decision problems that are *verifiable* in polynomial time.

- if a solution (also called *certificate*) is given, we can verify in polynomial time whether it's correct



Nondeterministic polynomial problems (NP)

Definition

The class **NP** consists of decision problems that are *verifiable* in polynomial time.

- if a solution (also called *certificate*) is given, we can verify in polynomial time whether it's correct
- examples:
 - shortest path: if we are given a path in a graph, we can easily verify if it's shorter than k
 - hamiltonian cycle: if we are given a cycle in a graph, we can easily verify if it passes through all vertices



Does $P=NP$?

- we know that $P \subseteq NP$
 - for all polynomial problems we can also verify the solution in polynomial time



Does $P=NP$?

- we know that $P \subseteq NP$
 - for all polynomial problems we can also verify the solution in polynomial time
- there are some difficult NP problems
 - we couldn't find a polynomial solution
 - we couldn't prove that such a solution doesn't exist



Does $P=NP$?

- we know that $P \subseteq NP$
 - for all polynomial problems we can also verify the solution in polynomial time
- there are some difficult NP problems
 - we couldn't find a polynomial solution
 - we couldn't prove that such a solution doesn't exist

Currently, $P \stackrel{?}{=} NP$ is a major unsolved problem in computer science.

- most theoretical computer scientists believe that $P \neq NP$



NP-complete problems (NPC)

Definition

An **NP-complete** problem is an NP problem at least as “difficult” as any problem in the class NP.



NP-complete problems (NPC)

Definition

An **NP-complete** problem is an NP problem at least as “difficult” as any problem in the class NP.

Theorem 34.4

If any NP-complete problem is solvable in polynomial time, then $P=NP$. Equivalently, if any problem in NP is not polynomial-time solvable, then no NP-complete problem is polynomial-time solvable.

- multiple proofs exist, none is trivial
- the theorem states that if we could solve a single NPC problem in polynomial time, then we could solve all of them in polynomial time



Reduction algorithms (1)

- when we encounter a new problem
 - either find a polynomial algorithm
 - or prove that it is NP-complete



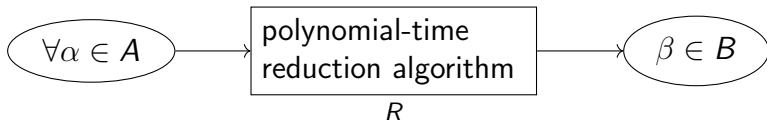
Reduction algorithms (1)

- when we encounter a new problem
 - either find a polynomial algorithm
 - or prove that it is NP-complete
- how do we prove that a problem is NPC?
 - by proving that it's at least as difficult as a known NPC problem



Reduction algorithms (2)

- let A be a known NPC problem and B a new problem that we want to prove is NPC



- by contradiction: if any instance α of A can be transformed in polynomial time to an instance β of B and B had a polynomial-time solution, then A would be polynomial
- the proof consists in finding the reduction algorithm R

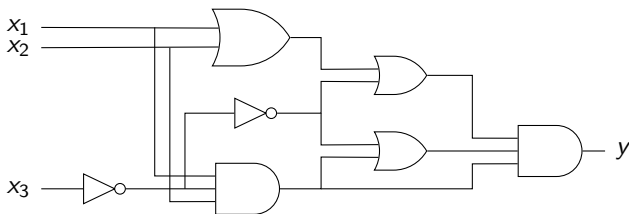


Agenda

- 1 Complexity classes
- 2 Classic NPC problems
- 3 Dealing with NPC problems in real world



Circuit satisfiability (CIRCUIT-SAT)



Can we assign 0/1 values to each input of a logic circuit such that the output is 1?

- optimization problem: find the actual assignment
- naive approach: try all 2^n possible assignments



Formula satisfiability (SAT)

$$\phi = ((x_1 \rightarrow x_2) \vee \overline{(\overline{x_1} \leftrightarrow x_3) \vee x_4}) \wedge \overline{x_2}$$

Can we assign 0/1 values to each input in order to satisfy a boolean formula (get the result 1)?



Formula satisfiability (SAT)

$$\phi = ((x_1 \rightarrow x_2) \vee \overline{(\overline{x_1} \leftrightarrow x_3)} \vee x_4) \wedge \overline{x_2}$$

Can we assign 0/1 values to each input in order to satisfy a boolean formula (get the result 1)?

Proof that $\text{SAT} \in \text{NP}$

- certificate: given an assignment of the inputs we can verify in polynomial time if the formula is satisfied
- reduction $\text{CIRCUIT-SAT} \leq_P \text{SAT}$: prove that any logic circuit can be transformed in polynomial time into a boolean formula



3-CNF satisfiability (3-CNF-SAT)

$$\phi = (x_1 \vee \overline{x_1} \vee \overline{x_2}) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\overline{x_1} \vee \overline{x_3} \vee \overline{x_4})$$

Can we assign 0/1 values to each input in order to satisfy a boolean formula in 3-conjunctive normal form?

- a boolean formula is in 3-CNF if it is expressed as an AND of clauses, each of them being the OR of exactly 3 literals (the occurrence of a variable or its negation)
- 3-CNF-SAT is NPC



3-CNF satisfiability (3-CNF-SAT)

$$\phi = (x_1 \vee \overline{x_1} \vee \overline{x_2}) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\overline{x_1} \vee \overline{x_3} \vee \overline{x_4})$$

Can we assign 0/1 values to each input in order to satisfy a boolean formula in 3-conjunctive normal form?

- a boolean formula is in 3-CNF if it is expressed as an AND of clauses, each of them being the OR of exactly 3 literals (the occurrence of a variable or its negation)
- 3-CNF-SAT is NPC
- however, 2-CNF-SAT is not NPC (solvable in polynomial time)



NP-complete graph problems

- **CLIQUE** A *clique* is a subset of nodes, each pair of which being connected by an edge. Does a given graph contains a clique of size k ?



NP-complete graph problems

- **CLIQUE** A *clique* is a subset of nodes, each pair of which being connected by an edge. Does a given graph contains a clique of size k ?
- **VERTEX-COVER** The *vertex cover* of $G = (V, E)$ is a subset of nodes $V' \subseteq V$ such that $\forall (u, v) \in E, u \in V'$ or $v \in V'$ (or both). Does a given graph contains a vertex cover of size k ?



NP-complete graph problems

- **CLIQUE** A *clique* is a subset of nodes, each pair of which being connected by an edge. Does a given graph contains a clique of size k ?
- **VERTEX-COVER** The *vertex cover* of $G = (V, E)$ is a subset of nodes $V' \subseteq V$ such that $\forall (u, v) \in E, u \in V'$ or $v \in V'$ (or both). Does a given graph contains a vertex cover of size k ?
- **HAMILTONIAN-CYCLE** Does a given graph contains a simple cycle that passes through each node?



NP-complete graph problems

- **CLIQUE** A *clique* is a subset of nodes, each pair of which being connected by an edge. Does a given graph contains a clique of size k ?
- **VERTEX-COVER** The *vertex cover* of $G = (V, E)$ is a subset of nodes $V' \subseteq V$ such that $\forall (u, v) \in E, u \in V'$ or $v \in V'$ (or both). Does a given graph contains a vertex cover of size k ?
- **HAMILTONIAN-CYCLE** Does a given graph contains a simple cycle that passes through each node?
- **TRAVELING-SALESPERSON** Can we find a hamiltonian cycle of cost smaller than k in a weighted graph?



The SUBSET-SUM problem

Given a finite set S of positive integers and an integer t , can we find a subset S' whose elements sum to t ?



The SUBSET-SUM problem

Given a finite set S of positive integers and an integer t , can we find a subset S' whose elements sum to t ?

- If $s = \sum_{x \in S} x$ we can solve this problem in $O(s)$ time and space using dynamic programming.
 - let $A^{(0)} = \emptyset$
 - compute $A^{(k)} = A^{(k-1)} \cup \{x_k\} \cup \{x + x_k : \forall x \in A^{(k-1)}\}$, where x_k is the k -th element of S



The SUBSET-SUM problem

Given a finite set S of positive integers and an integer t , can we find a subset S' whose elements sum to t ?

- If $s = \sum_{x \in S} x$ we can solve this problem in $O(s)$ time and space using dynamic programming.
 - let $A^{(0)} = \emptyset$
 - compute $A^{(k)} = A^{(k-1)} \cup \{x_k\} \cup \{x + x_k : \forall x \in A^{(k-1)}\}$, where x_k is the k -th element of S
- unfortunately, in the general case s can take any value, like 2^n or even 2^{2^n}



Agenda

- 1 Complexity classes
- 2 Classic NPC problems
- 3 Dealing with NPC problems in real world



Make sure the problem is really NPC

- many problems “look” very similar to NPC problems but have known polynomial algorithms
 - 2-SAT vs. 3-SAT
 - EULERIAN-CYCLE vs. HAMILTONIAN-CYCLE
 - shortest path vs. longest path
 - MST / SSSP vs. TSP



Make sure the problem is really NPC

- many problems “look” very similar to NPC problems but have known polynomial algorithms
 - 2-SAT vs. 3-SAT
 - EULERIAN-CYCLE vs. HAMILTONIAN-CYCLE
 - shortest path vs. longest path
 - MST / SSSP vs. TSP
- particular instances of NPC problems are not necessarily NPC
 - SUBSET-SUM, KNAPSACK $\in P$ if the sum of elements is small
 - deciding if the vertices of a graph can be colored with k colors is NPC, but for $k = 2$ it is polynomial (bipartite graphs)



Avoid implementing exponential algorithms

- exponential algorithms only work for very small inputs
- assume you have an $O(2^n)$ algorithm that runs in 1 minute for $n = 10$
 - it will take 17h for $n = 20$
 - ~ 2 years for $n = 30$
 - ~ 2000 years for $n = 40$
- even if the CPU speed increases k times or you can use k CPU cores, you can run your algorithm in the same time for $n' = n + \log k$ (see lecture #1)



Avoid implementing exponential algorithms

- exponential algorithms only work for very small inputs
- assume you have an $O(2^n)$ algorithm that runs in 1 minute for $n = 10$
 - it will take 17h for $n = 20$
 - ~ 2 years for $n = 30$
 - ~ 2000 years for $n = 40$
- even if the CPU speed increases k times or you can use k CPU cores, you can run your algorithm in the same time for $n' = n + \log k$ (see lecture #1)
- also, avoid wasting your time trying to solve NPC problems in polynomial time (unless you're a top theoretical computer scientist)



Consider using approximation algorithms

- in real world, an imperfect solution is better than no solution



Consider using approximation algorithms

- in real world, an imperfect solution is better than no solution
- example: you have a set of objects that need to be packed in as few boxes as possible (BIN-PACKING)
 - run an algorithm for 1000 years that will give the optimal solution (smallest number of boxes)
 - place each object in the first box that fits and risk using more boxes than optimal, but no more than twice the optimal number of boxes



Approximation algorithms

Definition

An algorithm for a problem has an **approximation ratio** of $\rho(n)$ if, for any input of size n , the cost C of the solution produced by the algorithm is within a factor $\rho(n)$ of the cost C^* of an optimal solution.

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n)$$



Approximation algorithms

Definition

An algorithm for a problem has an **approximation ratio** of $\rho(n)$ if, for any input of size n , the cost C of the solution produced by the algorithm is within a factor $\rho(n)$ of the cost C^* of an optimal solution.

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n)$$

- for some problems ρ is constant, for others it grows with n



Approximation algorithms

Definition

An algorithm for a problem has an **approximation ratio** of $\rho(n)$ if, for any input of size n , the cost C of the solution produced by the algorithm is within a factor $\rho(n)$ of the cost C^* of an optimal solution.

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n)$$

- for some problems ρ is constant, for others it grows with n
- there are also approximation schemes where $\rho(n) = 1 + \epsilon$
 - the complexity grows as ϵ decreases
 - we trade computation time for a better approximation



Approximating VERTEX-COVER

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```




Approximating VERTEX-COVER

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```

- let A be the set of edges picked at line 4



Approximating VERTEX-COVER

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```

- let A be the set of edges picked at line 4
- $|A| \leq |C^*|$ (any solution must include at least one endpoint for each edge in A)



Approximating VERTEX-COVER

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```

- let A be the set of edges picked at line 4
- $|A| \leq |C^*|$ (any solution must include at least one endpoint for each edge in A)
- $|C| = 2|A|$



Approximating VERTEX-COVER

APPROX-VERTEX-COVER(G)

```
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```

- let A be the set of edges picked at line 4
- $|A| \leq |C^*|$ (any solution must include at least one endpoint for each edge in A)
- $|C| = 2|A|$
- $\Rightarrow |C| \leq 2|C^*| \Rightarrow \rho(n) = 2$



Other approximation results

- MAX-3-CNF-SAT (maximize the number of satisfied clauses)
 $\frac{8}{7}$ -approximable (randomized)
- TRAVELING-SALESPERSON
 - in a graph with triangle inequality, we have a 2-approximation
 - in a general graph, TSP is not approximable
- SUBSET-SUM is $(1 + \epsilon)$ -approximable with an algorithm that is polynomial both in n and $1/\epsilon$ (FPTAS)



The NP compendium

<https://www.csc.kth.se/~viggo/problemlist/>

- very useful for searching NP optimization problems
- each problem has
 - a brief description
 - good news - best optimization algorithm
 - bad news - proof of inapproximability (within some constant)
 - comments



Bibliography

- Cormen, Thomas H., et al., *"Introduction to algorithms."*, MIT press, 2009, cap. 34 & cap. 35